Enunciado

Trabajo Práctico - Parte 2

Introducción y Contexto General

En la primera etapa del trabajo práctico se establecieron los cimientos del sistema mediante el modelado de las naves, sus recursos y su tripulación. En esta segunda parte, el objetivo es dar vida a ese modelo: **pondremos la nave en movimiento a través del espacio, incorporaremos una interfaz de control en tiempo real y sumaremos nuevos sistemas a bordo.**

El funcionamiento general de esta nueva versión se organiza en torno a tres ejes principales que conforman la operatoria completa de la nave:

Control de Acceso al Sistema (Registro de Usuarios)

Antes de tomar el mando y operar los sistemas de la nave, es necesario identificar al operador a cargo. El sistema incorpora una instancia de acceso donde cada usuario debe registrarse o iniciar sesión con sus credenciales. Una vez validada su identidad, el operador accede al panel central, desde el cual podrá configurar el tipo de nave, conformar la tripulación, consultar sus salarios y gestionar las operaciones de vuelo.

La Travesía Espacial (Misiones)

Con la nave tripulada y en condiciones operativas, comienza la exploración del universo. A través del asistente de comandos, se podrán iniciar las distintas misiones disponibles: acudir al auxilio de una nave aliada para transferirle suministros, dirigirse a coordenadas remotas para recolectar gemas, o emprender el retorno a una base segura para reabastecer recursos y reparar el desgaste acumulado.

Sin embargo, el trayecto hacia el objetivo no es lineal ni está despejado. A medida que la nave calcula su ruta con los radares y avanza paso a paso hacia su destino, el universo a su alrededor cambia dinámicamente en cada turno. En su camino, la nave deberá afrontar distintos imprevistos:

- **Obstáculos espaciales:** campos de asteroides que impactan contra el casco y reducen las reservas de energía.
- **Puntos de asistencia:** estaciones de carga que permiten recuperar energía y combustible en medio del viaje.
- **Encuentros con otras naves:** cruces en una misma coordenada que, dependiendo del tipo de cada nave, pueden resolverse mediante un pacto de paz o derivar en un enfrentamiento directo donde la derrota implica la destrucción total de la nave y el fracaso de la misión.

Todo el progreso de la travesía se monitorea en vivo desde la pantalla de control, permitiendo observar el movimiento por el mapa, la variación de los recursos, los registros de la bitácora y el informe final con el balance de cada misión.

El Visor del Universo Paralelo

De forma paralela a la navegación principal, en una sección apartada de la nave existe un visor que permite observar la actividad de otra dimensión. Los tripulantes utilizan este dispositivo durante sus tiempos libres para seguir la interacción entre **Generadoras de Energía** y **Naves Desconocidas** en torno a una única **Plataforma de Carga**.

En dicho universo, las generadoras producen baterías y buscan depositarlas en la plataforma, mientras que las naves acuden a ella para retirarlas y abastecerse. La particularidad del sistema radica en que la plataforma tiene una capacidad limitada y solo admite que una entidad opere sobre ella a la vez: si se llena, las generadoras deben aguardar su turno para depositar; si se vacía, las naves deben esperar hasta que se repongan suministros.

Para hacer la observación más dinámica, el capitán incorporó un panel con dos controles que permiten, en cualquier momento, introducir nuevas naves o nuevas generadoras de energía dentro de ese universo, reflejando en tiempo real en la pantalla del visor cómo se coordinan, esperan y compiten por el acceso a la plataforma.

Misiones

Las misiones disponibles son las mismas que las planteadas en la Parte 1, pero en esta etapa profundizaremos en la lógica de su ciclo de vida y ejecución en el tablero.

`M-01` — Intercepción y asistencia:

El objetivo es alcanzar a una nave aliada que solicita auxilio y entregarle recursos (energía o combustible) para que pueda sobrevivir. Estos recursos se transfieren únicamente cuando ambas naves se encuentran en la misma posición.

`M-02` — Recolección:

El objetivo es llegar hasta una coordenada donde se encuentran riquezas espaciales. Al arribar al destino, la nave recolecta gemas que le servirán para su travesía por el universo.

`M-03` — Retorno a punto seguro:

El objetivo es regresar a la base de origen (punto de partida del recorrido) para recargar recursos y reparar el desgaste de la nave.

Ciclo de Vida de una Misión

Todas las misiones comparten una estructura de ejecución fija compuesta por cuatro fases secuenciales, aunque ciertos pasos específicos varían según el tipo de misión.

Requerimiento de Diseño — Patrón Template Method:

El ciclo de vida de las misiones debe implementarse aplicando el patrón Template Method. La clase base deberá definir el esqueleto del algoritmo (el método plantilla que invoca a `preparar`, `ejecutar`, `evaluar` y `cerrar` en orden), delegando en las subclases los pasos que dependen de cada tipo de misión particular.

A continuación se detalla el comportamiento esperado para cada fase:

Fase 1: Preparar (`preparar`)

1. **Verificación inicial:** Comprobar que la nave cuente con los recursos mínimos necesarios para iniciar la misión.
2. **Cálculo de ruta:** Utilizar el **radar de largo alcance** para calcular el camino hacia el objetivo y almacenar la ruta resultante.

Fase 2: Ejecutar (`ejecutar`)

1. **Consumo de despegue:** Descontar los recursos iniciales requeridos para poner en marcha la misión.
2. **Ciclo de travesía:** Mientras la nave siga con vida y aún no haya alcanzado la coordenada de destino, se ejecuta un bucle por turnos dividido en dos etapas:
   - **A. Turno de la Nave:**
     1. **Inspección de posición actual:** Consultar si existe algún `ElementoDelUniverso` en la coordenada actual de la nave. En caso afirmativo, **interactuar con dicho elemento** y aplicar las consecuencias correspondientes (ganancia o pérdida de recursos, incremento de desgaste o destrucción). *Nota: Si el desgaste alcanza el valor de 100, la energía queda en 0 o el combustible queda en 0, la nave se destruye y la tripulación muere, dando por fallida la misión de inmediato.*
     2. **Escaneo:** Activar el **radar de corto alcance** para evaluar el entorno inmediato.
     3. **Planificación y movimiento:** Determinar el próximo movimiento y ejecutar el salto hacia la siguiente coordenada.
   - **B. Turno del Tablero (`Destino`):** El tablero mueve de forma aleatoria las piezas/elementos presentes en el universo.
3. **Verificación en destino:** Una vez alcanzada la coordenada objetivo, si la misión exige entregar recursos adicionales, se debe verificar que la nave **aún conserve dichos recursos** tras el viaje. Si los consumió en el trayecto y ya no los posee, la misión se considera **fallida** *(nota: el descuento efectivo no se realiza en este paso, sino en el cierre)*.

Nota técnica:

En cada iteración del bucle de turnos se deberá invocar un `Thread.sleep(int)` con una pausa breve, de modo que la misión no se resuelva instantáneamente y el usuario pueda observar el avance paso a paso en la vista del tablero.

Fase 3: Evaluar (`evaluar`)

- Confeccionar el **Informe Final de Misión**, registrando si la misión resultó **exitosa o fallida**, junto con el balance detallado de los recursos ganados, consumidos o perdidos durante el trayecto.

Fase 4: Cerrar (`cerrar`)

- Impactar de forma definitiva en el estado de la nave los recursos ganados o entregados al completar la misión.

Elementos del Universo e Interacciones

Durante su travesía por el tablero, la nave puede coincidir en una misma coordenada con distintos elementos del espacio, los cuales deben implementar la interfaz ElementoDelUniverso:

1. **Otras Naves:** Exploradoras, Cargueras o de Combate.
2. **Obstáculos:** Asteroides.
3. **Ayudas:** Estaciones de carga.

A. Encuentro con Otras Naves (Batallas)

Cuando dos naves coinciden en una posición, se enfrentan en batalla. El resultado del combate **no depende de las estadísticas ni de los recursos actuales**, sino exclusivamente del **tipo de cada nave**. La resolución de los encuentros entre la nave principal y las naves enemigas **debe implementarse utilizando *Double Dispatch***.

	Nave Enemiga		
Nave Propia	**vs. Exploradora**	**vs. Carguera**	**vs. Combate**
Exploradora	Pacto de paz	Gana	Pierde
Carguera	Pierde	Pacto de paz	Gana
Combate	Gana	Pierde	Pacto de paz

Si la nave del jugador pierde el enfrentamiento, **es destruida junto con toda su tripulación**, provocando el fallo inmediato de la misión.

B. Encuentro con Obstáculos (Asteroides)

- Colisionar con un asteroide descuenta un **monto fijo de energía**.
- Si como consecuencia del impacto la energía de la nave llega a 0 (o menos), la nave queda inoperativa y la misión **falla inmediatamente**.

C. Encuentro con Ayudas (Estaciones de Carga)

- Las estaciones espaciales permiten reabastecer a la nave, recargando su **combustible y energía** ya sea en un monto fijo o en un porcentaje sobre sus recursos.

Destino y Tablero

La clase `Destino` actúa como el controlador del entorno espacial y será la encargada de mover aleatoriamente los elementos del universo a través del tablero en cada turno.

- **Provisión de la cátedra:** La clase `Destino` será provista por la cátedra para que la integren directamente en sus proyectos (su documentación detallada se entregará próximamente).
- **Responsabilidad del grupo:** Ustedes deberán diseñar e implementar todas las clases concretas de los elementos espaciales con la interfaz `ElementoDelUniverso`.

Concurrencia

En una sección recóndita de la nave existe un visor hacia un universo paralelo. En sus ratos libres, la tripulación se entretiene observando cómo interactúan **Naves Desconocidas** y **Generadoras de Energía** alrededor de una única **Plataforma de Carga**.

El capitán, al notar el entusiasmo de la tripulación, instaló un panel de control con dos botones que permiten intervenir en ese universo en tiempo real:

1. **Crear Nave Consumidora**
2. **Crear Generadora de Energía**

La particularidad de esta **Plataforma de Carga** es que posee capacidad limitada y **acceso exclusivo**: en un instante dado, solo puede ser utilizada por una única entidad a la vez (o bien una generadora depositando, o bien una nave retirando).

A la tripulación le divierte adivinar quién tomará el control de la plataforma a continuación y ver qué ocurre cuando una generadora intenta depositar una batería en una plataforma llena o una nave intenta retirar cuando está vacía. Cosas de extraterrestres.

Especificación Técnica de Concurrencia

> **Requerimiento de Diseño — Patrón Productor-Consumidor con Monitores:**
> Este módulo debe resolverse aplicando el problema clásico de **Productor-Consumidor** mediante **Monitores en Java** (`synchronized`, `wait()`, `notifyAll()`). La creación de nuevas naves y generadoras debe poder realizarse dinámicamente en tiempo de ejecución mientras la simulación está activa.

1. Productores (`GeneradoraDeEnergia`)

Cada generadora se ejecuta en su **propio hilo (`Thread`)** y repite el siguiente ciclo:

1. Produce una batería.
2. Solicita acceso a la `PlataformaDeCarga` para depositarla.
3. Si la plataforma está llena, **espera (`wait`)** hasta que haya lugar disponible.
4. Cuando hay espacio, deposita la batería.

5. **Notifica (`notifyAll`)** a los demás hilos en espera que el estado de la plataforma cambió.

2. Consumidores (`NaveConsumidora`)

Cada nave del universo paralelo se ejecuta en su **propio hilo (`Thread`)** y repite el siguiente ciclo:

1. Solicita una batería.
2. Intenta retirarla de la `PlataformaDeCarga`.
3. Si no hay baterías disponibles, **espera (`wait`)** hasta que alguna generadora deposite una.
4. Cuando hay disponibilidad, retira y consume la batería.
5. **Notifica (`notifyAll`)** a los demás hilos en espera que el estado de la plataforma cambió.

3. Recurso Compartido (`PlataformaDeCarga`)

Coordina la exclusión mutua y la sincronización entre productores y consumidores mediante:

- Métodos declarados como `synchronized`.
- Evaluación de condiciones de guarda (depósito lleno / depósito vacío) dentro de **bucles `while`** para volver a comprobar la condición tras despertar.
- Invocaciones a `wait()` y `notifyAll()`.

4. Registro de Eventos en el Visor

La simulación debe informar en tiempo real a la interfaz gráfica los eventos que van ocurriendo (aplicando el patrón **Observer**). Ejemplos de eventos a mostrar:

```
› "Batería producida por [Generadora X]."
› "[Generadora X] esperando a que se libere espacio en la Plataforma."
› "[Nave Y] solicitando batería."
› "[Nave Y] esperando por falta de baterías en la Plataforma."
› "Batería retirada y utilizada por [Nave Y]."
› "¡Ingresó una nueva [Nave / Generadora] al universo paralelo!"
```

(Pueden agregar mensajes adicionales para enriquecer la simulación y entretener a su tripulación).

Persistencia

Se requiere identificar quién utiliza el sistema mediante un mecanismo de **Inicio de Sesión (Login)** y **Registro de Usuarios**.

> **Requerimiento de Diseño — Patrones DAO y DTO:**
>
> La capa de persistencia debe implementarse utilizando los patrones **DAO (*Data Access Object*)** y **DTO (*Data Transfer Object*)**.

Alcance de la persistencia:

Únicamente se deben persistir las credenciales de los usuarios (**nombre de usuario y contraseña**) para permitirles registrarse y volver a iniciar sesión en futuras ejecuciones. **No se debe persistir ninguna otra entidad del sistema** (naves, tripulantes, misiones, etc.).

Vistas

Pantallas y Secciones Requeridas

Pueden optar por organizar la interfaz en una ventana principal con paneles/pestañas o en múltiples ventanas, siempre que se contemplen las siguientes vistas:

1. **Acceso al Sistema:**
   - **Vista de Login:** Permite ingresar nombre de usuario y contraseña existentes.
   - **Vista de Registro:** Permite dar de alta un nuevo usuario con nombre y contraseña.
2. **Sección de Creación:**
   - Permite seleccionar el tipo de Nave e instanciarla.
   - Permite crear tripulantes y asignarlos a la nave.
3. **Sección de Tripulantes:**
   - Muestra el listado de tripulantes con su información detallada y el **salario** correspondiente a cada uno.
4. **Sección de Control de la Nave:**
   - Estado actual de los **recursos de la nave**.
   - **Bitácora** de eventos.
   - Historial de **informes finales** de misiones.
   - Controles para **iniciar una misión** y visualización del **tablero en tiempo real** mientras la nave avanza por el universo *(actualización automática)*.
5. **Sección del Visor Recóndito (Concurrencia):**
   - Botones para agregar Naves Consumidoras y Generadoras de Energía en vivo.
   - Consola/panel que muestra en tiempo real el flujo de eventos (solo texto) de la Plataforma de Carga *(actualización automática)*.

Requerimiento de Diseño — Patrones MVC y Observer-Observable:

La comunicación entre la interfaz de usuario y la lógica de negocio debe separar responsabilidades combinando **MVC** (para acciones del usuario) y **Observer** (para actualizaciones automáticas del modelo hacia la vista).

Flujo para interacciones iniciadas por el usuario (MVC):

- El usuario interactúa con un componente de la **Vista** (ej. presiona un botón).

- Un `ActionListener` captura el evento en la Vista y delega la acción al **Controlador**.
- El **Controlador** invoca la operación correspondiente en el **Modelo**.
- El **Modelo** procesa la lógica de negocio y retorna el resultado (o cambia su estado interno).
- El **Controlador** evalúa el resultado y coordina la respuesta con la **Vista**.

Entregables

- Repositorio del código completo con README.
- Informe que continúe lo presentado en la parte 1 y añada los nuevos contenidos.

Nota: Luego de la entrega de la segunda parte del trabajo práctico, deberán hacer una presentación sobre su solución. Informaremos más detalles de la modalidad a la brevedad.